Introdução à Computação
listas encadeadas

Alexandre Rademaker

listas encadeadas I

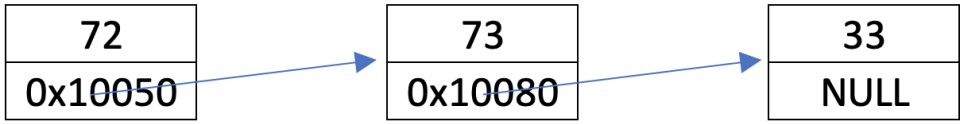
Com as listas encadeadas (linked lists) podemos armazenar valores em diferentes posições de memória (não contínuas).

Com isso, podemos acrescentar novos valores na lista continuamente, sem necessidade de estabelecer um limite 'a priori' nem realocação quando o limite é atingido.

listas encadeadas II

Para cada valor, alocamos também o endereço do próximo valor na lista, o par é uma cons cell.

endereço	valor
0x10000	?
0x10004	72
0x10008	0x10050
...	?
0x10050	73
0x10054	0x10080
...	?
0x10080	33
0x10084	NULL
...	?



listas encadeadas III

Em C, podemos implementar uma **cons cell** (node) com o código.

```
1 typedef struct node
2 {
3   int number;
4   struct node *next;
5 } t_node;
```

Trata-se de uma estrutura de dados recursiva.

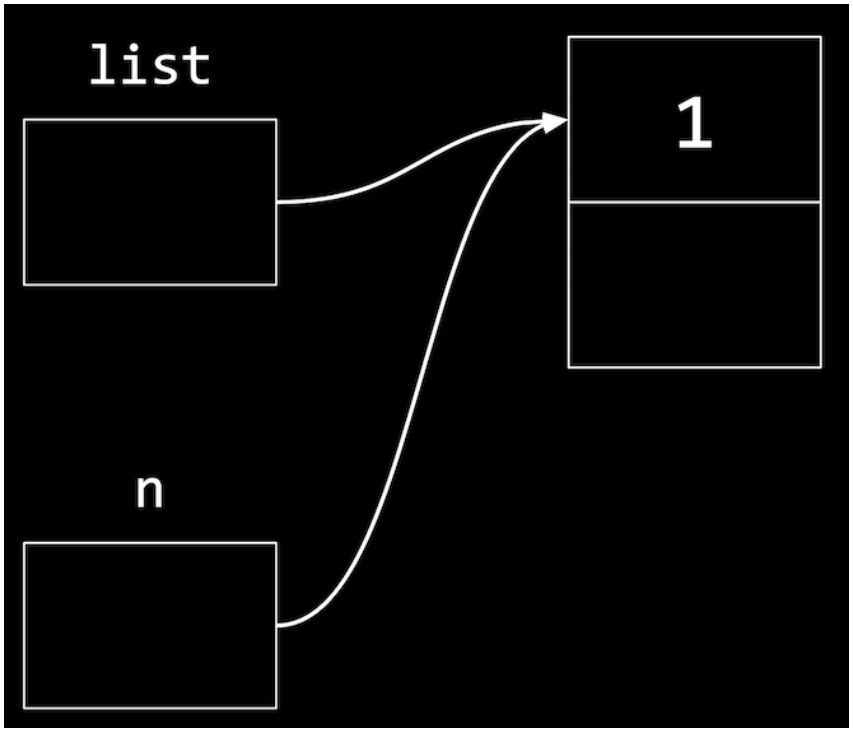
listas encadeadas IV

```
// lista vazia
t_node *list = NULL;

t_node *n = malloc(sizeof(t_node));
if (n != NULL) {
  n->number = 5;  // same as (*n).number
  n->next = NULL;
}

list = n;
```

A partir da lista vazia, adicionamos o número 5 na frente da lista.

listas encadeadas V

Graficamente, podemos representar o resultado do código anterior com



lista encadeada na memória

```
typedef struct node
{
    int number;
    struct node *next;
}
node;
```

lista encadeada na memória

```
node *list;
```

lista encadeada na memória

lista encadeada na memória

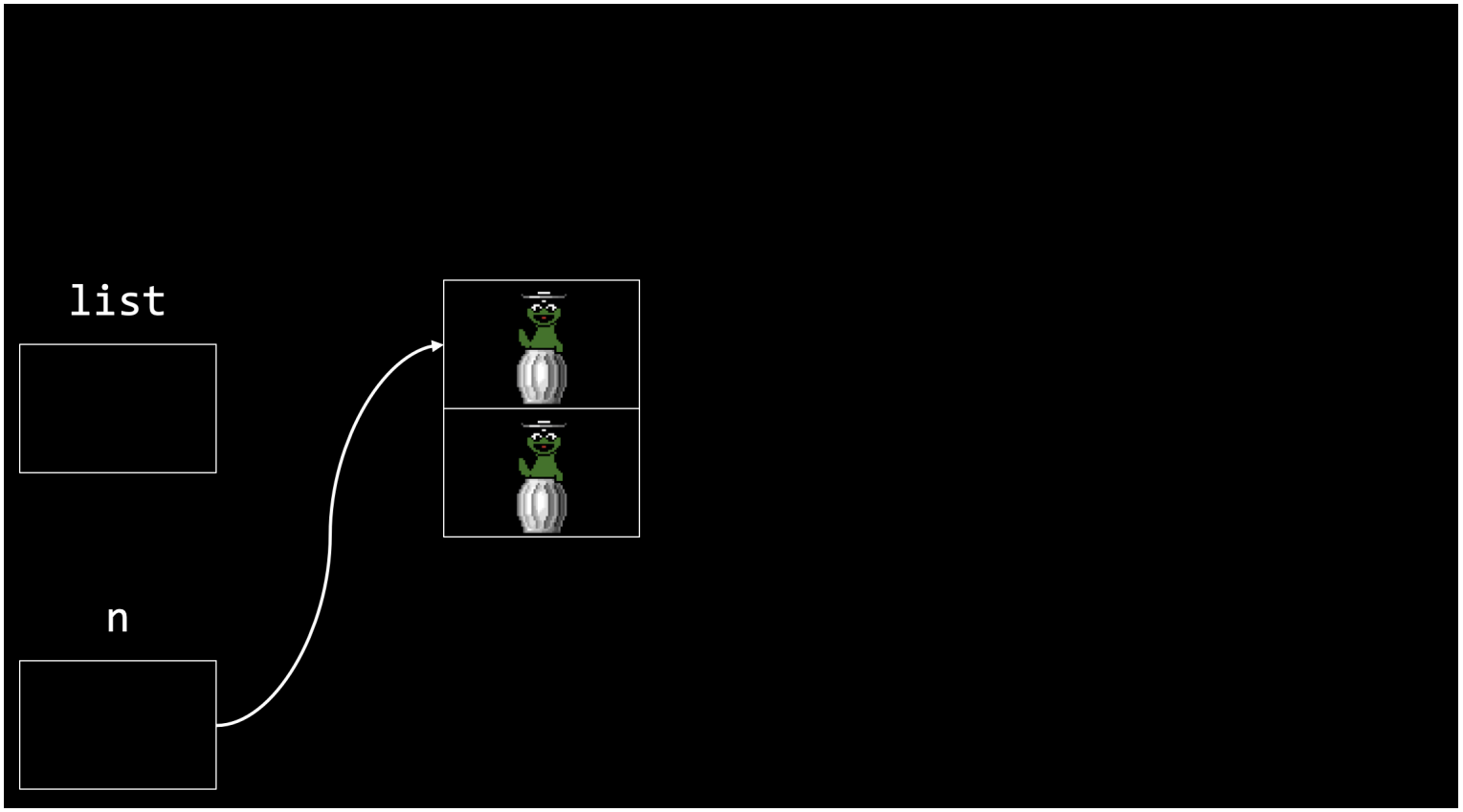
```
node *list = NULL;
```

lista encadeada na memória

lista encadeada na memória

```
node *n = malloc(sizeof(node));
```

lista encadeada na memória



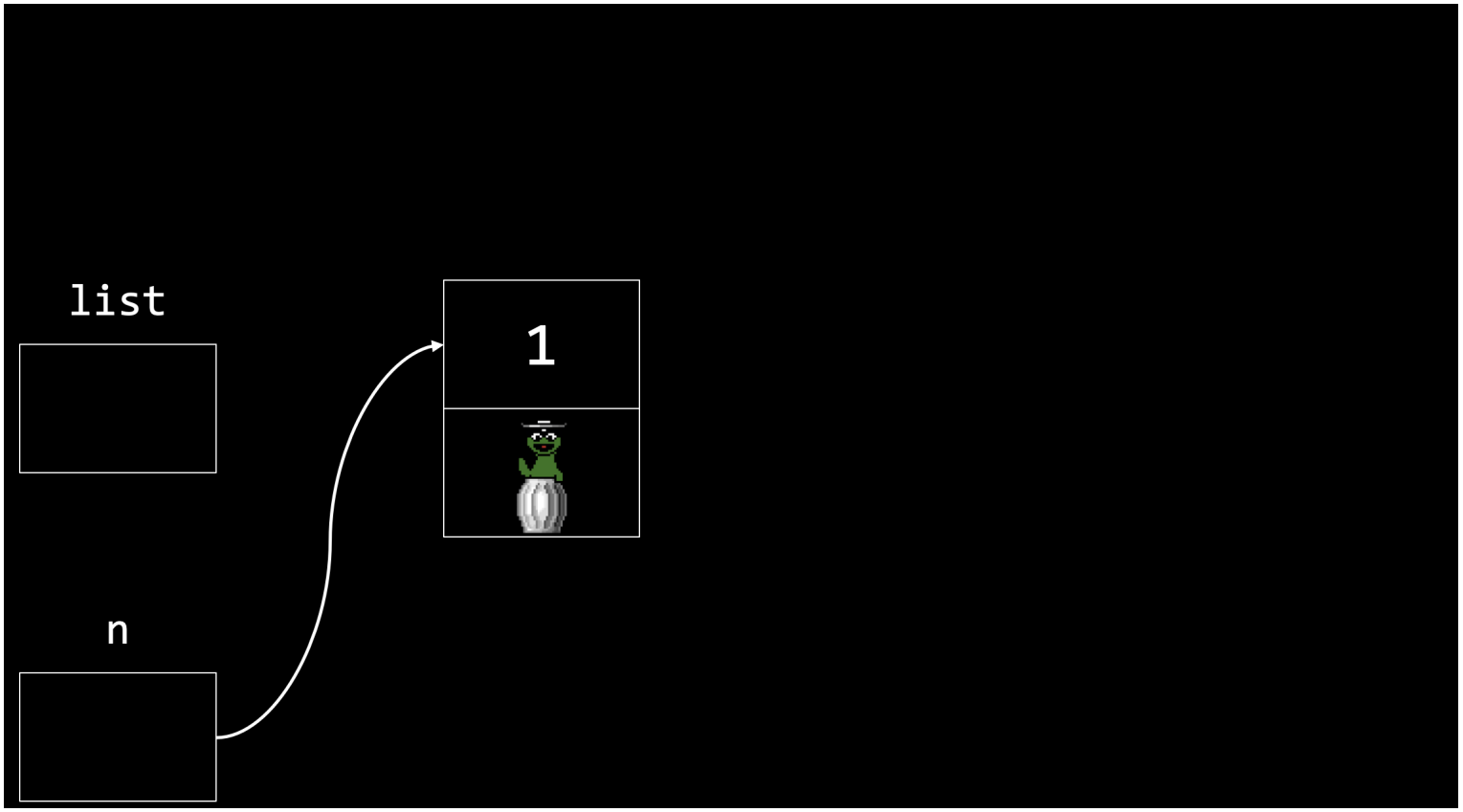
lista encadeada na memória

```
if (n != NULL)
{
    (*n).number = 1;
}
```

lista encadeada na memória

```
if (n != NULL)
{
    n->number = 1;
}
```

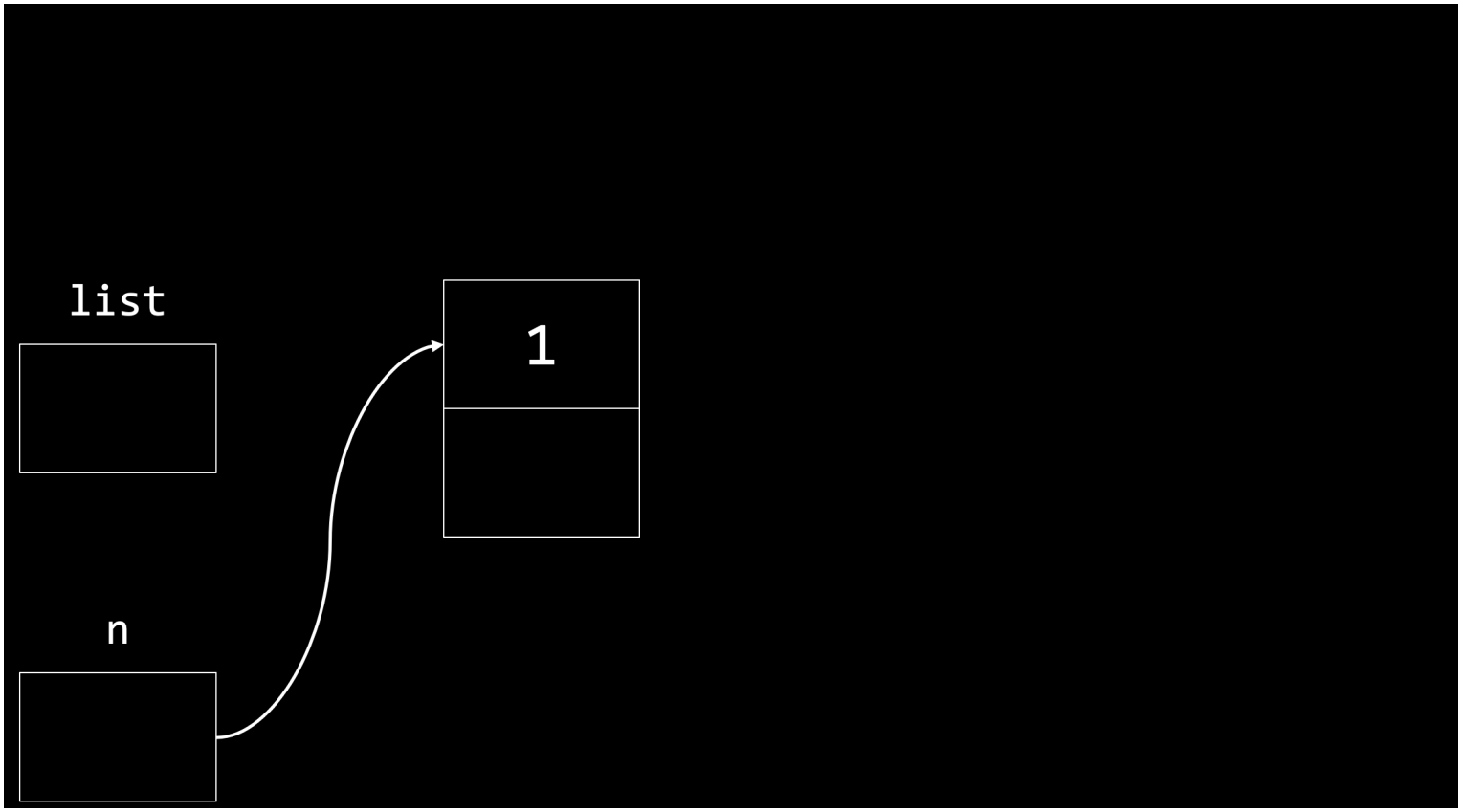
lista encadeada na memória



lista encadeada na memória

```
if (n != NULL)
{
    n->next = NULL;
}
```

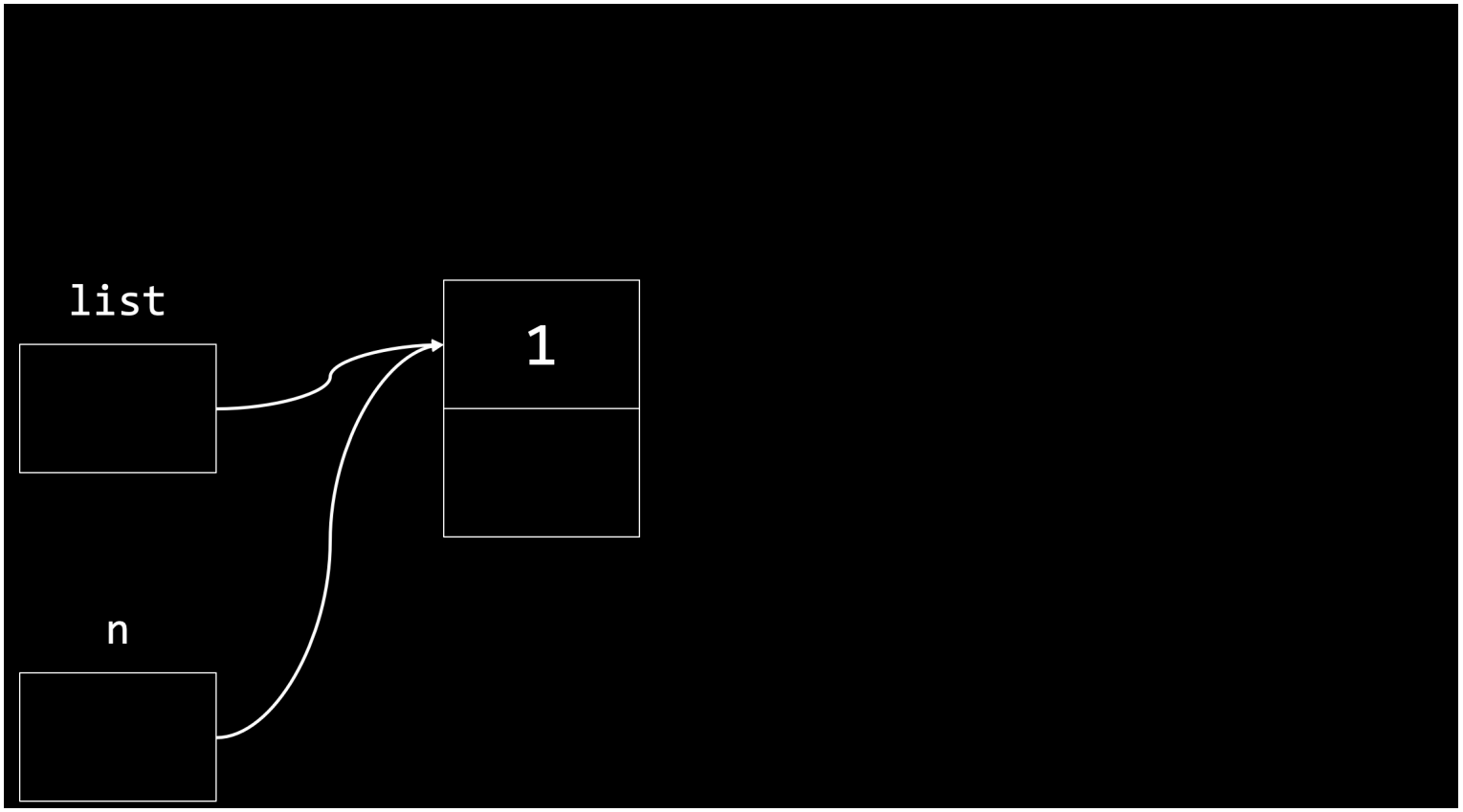
lista encadeada na memória



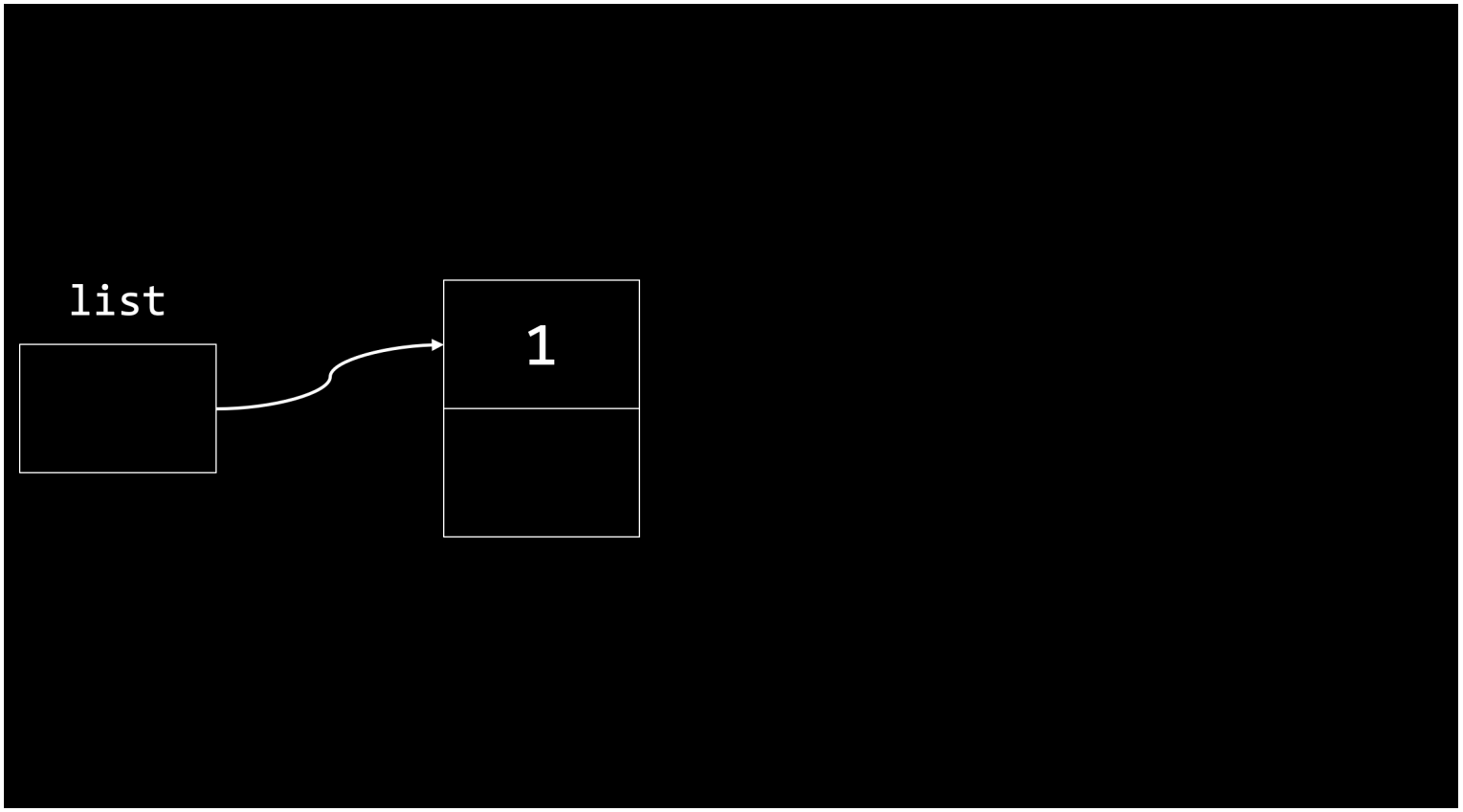
lista encadeada na memória

```
list = n;
```

lista encadeada na memória



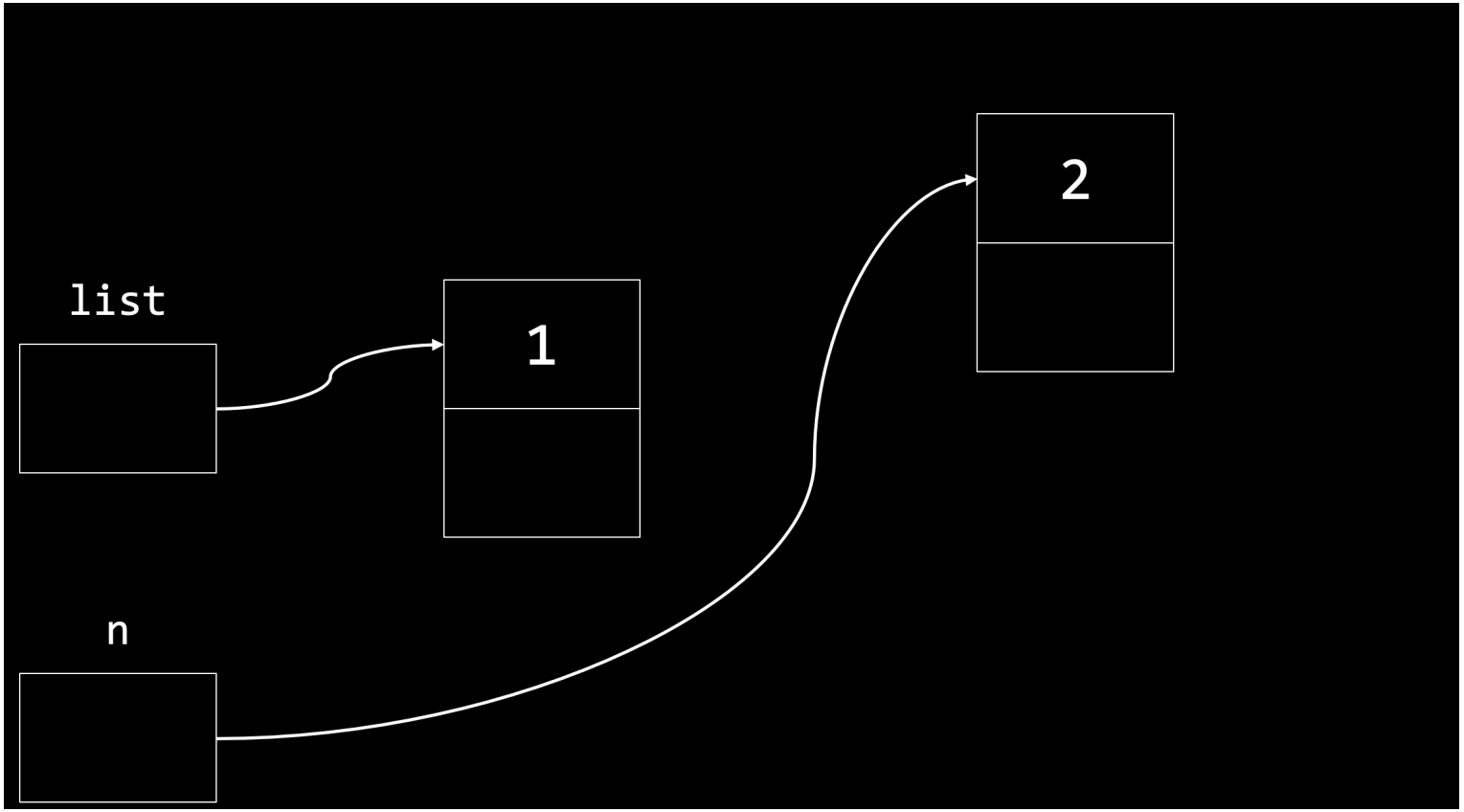
lista encadeada na memória



lista encadeada na memória

```
node *n = malloc(sizeof(node));
if (n != NULL)
{
    n->number = 2;
    n->next = NULL;
}
```

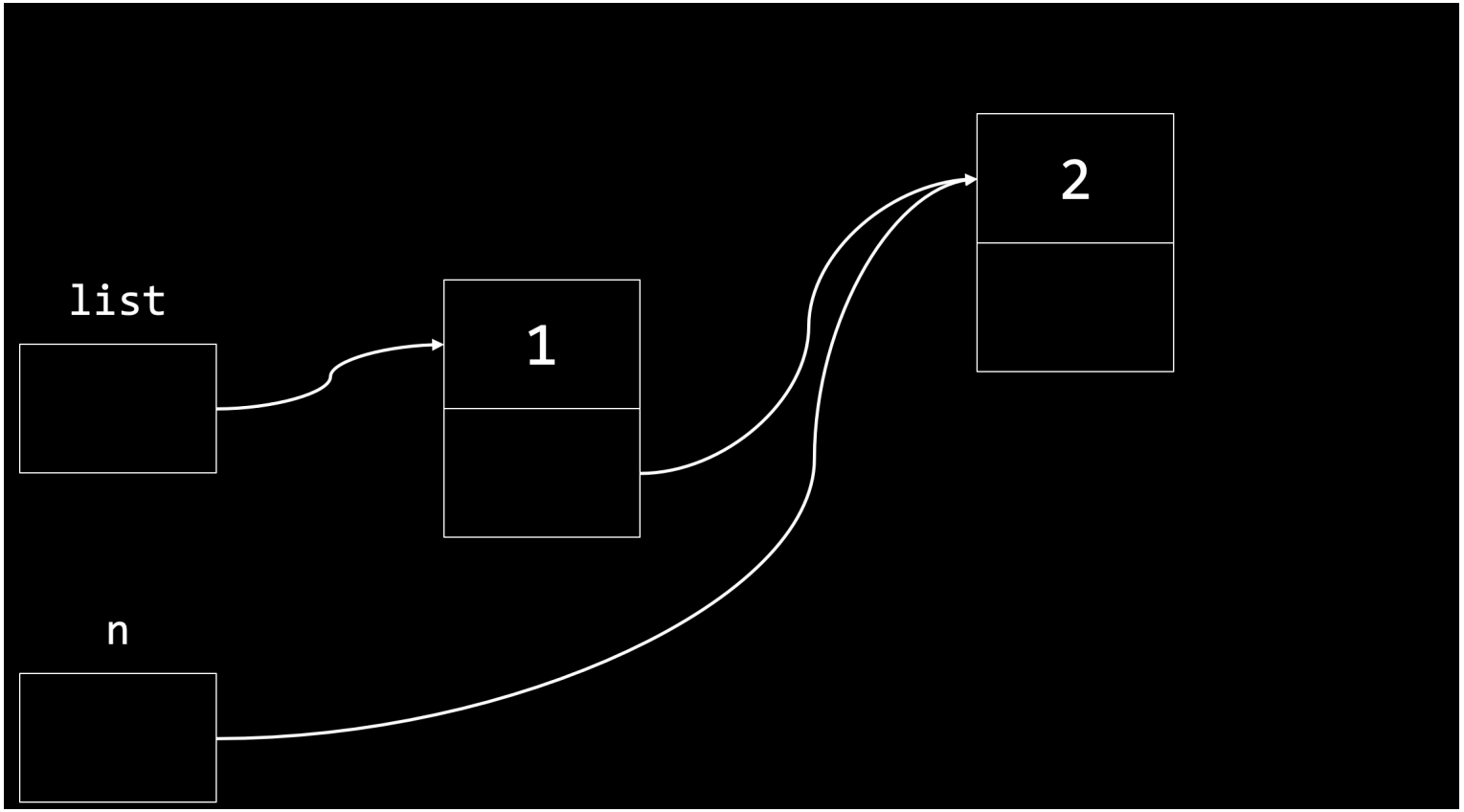
lista encadeada na memória



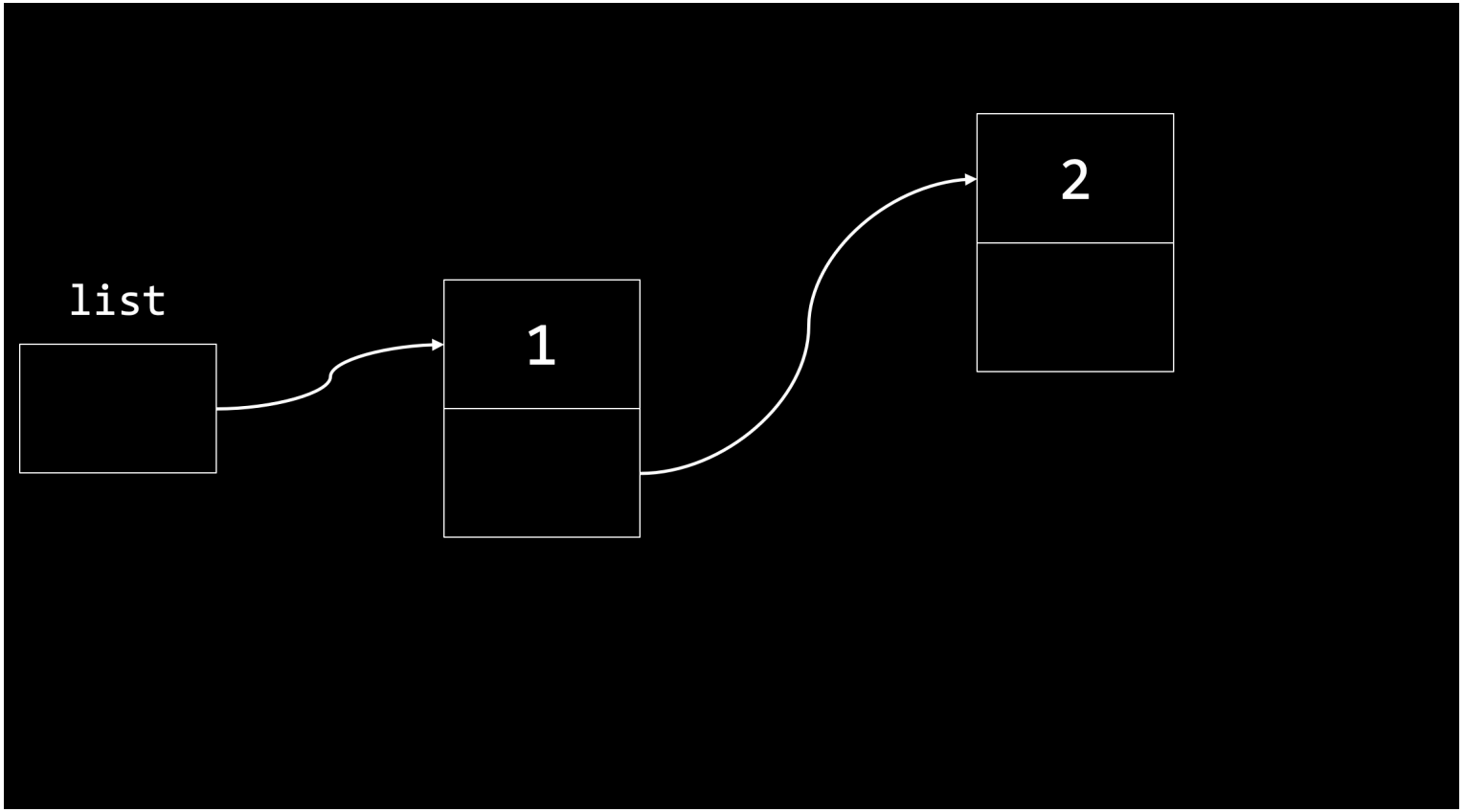
lista encadeada na memória

```
list->next = n;
```

lista encadeada na memória



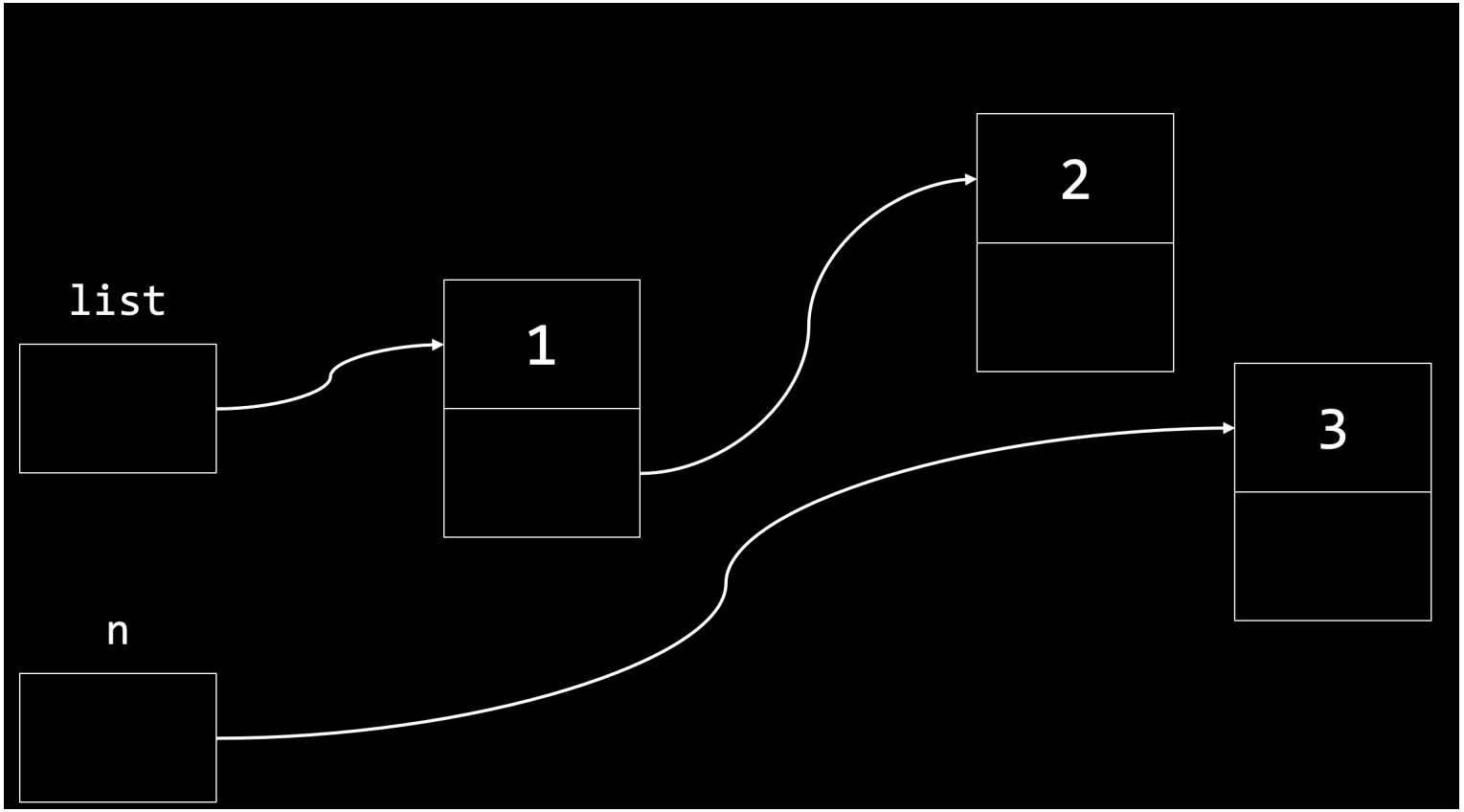
lista encadeada na memória



lista encadeada na memória

```
node *n = malloc(sizeof(node));
if (n != NULL)
{
    n->number = 3;
    n->next = NULL;
}
```

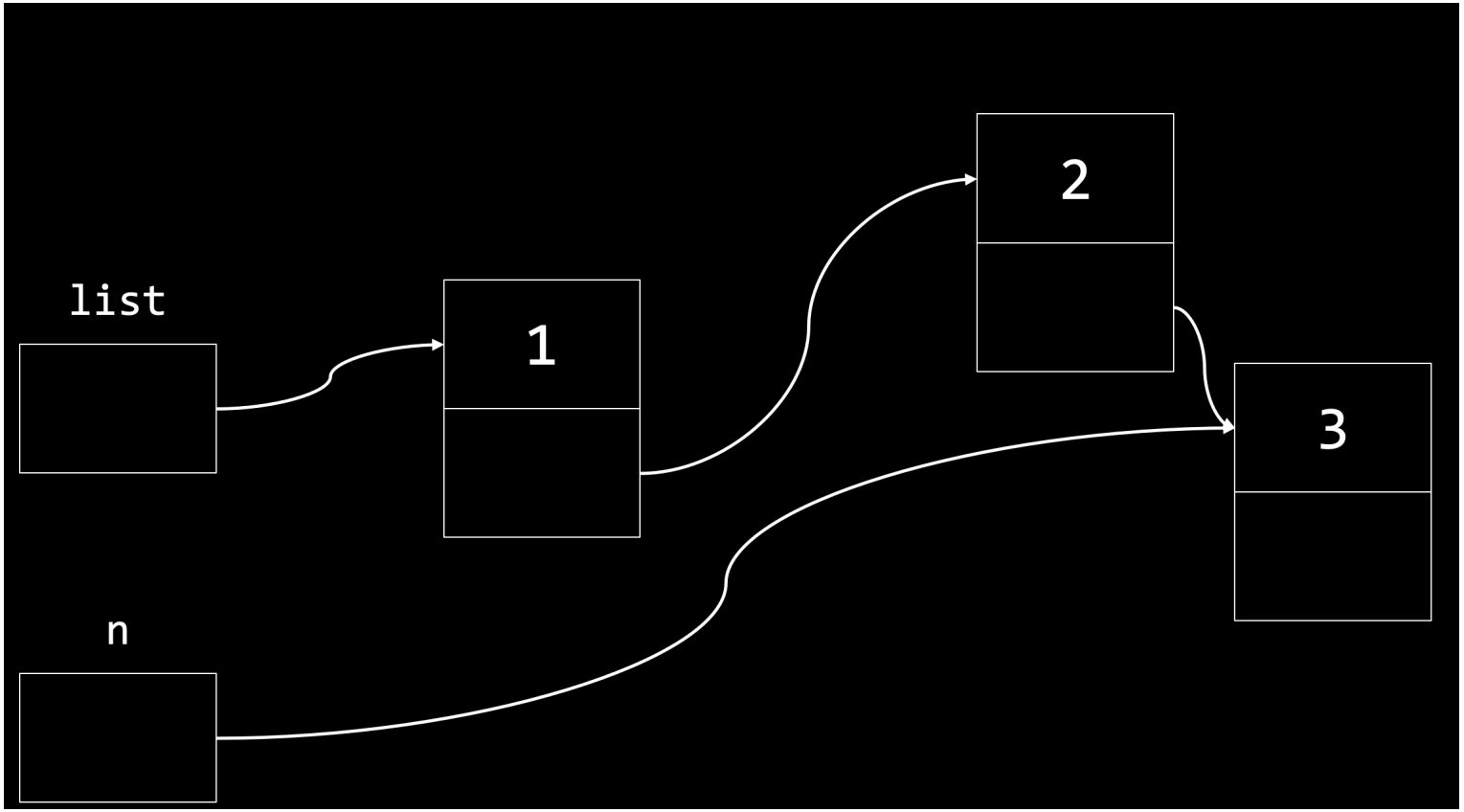
lista encadeada na memória



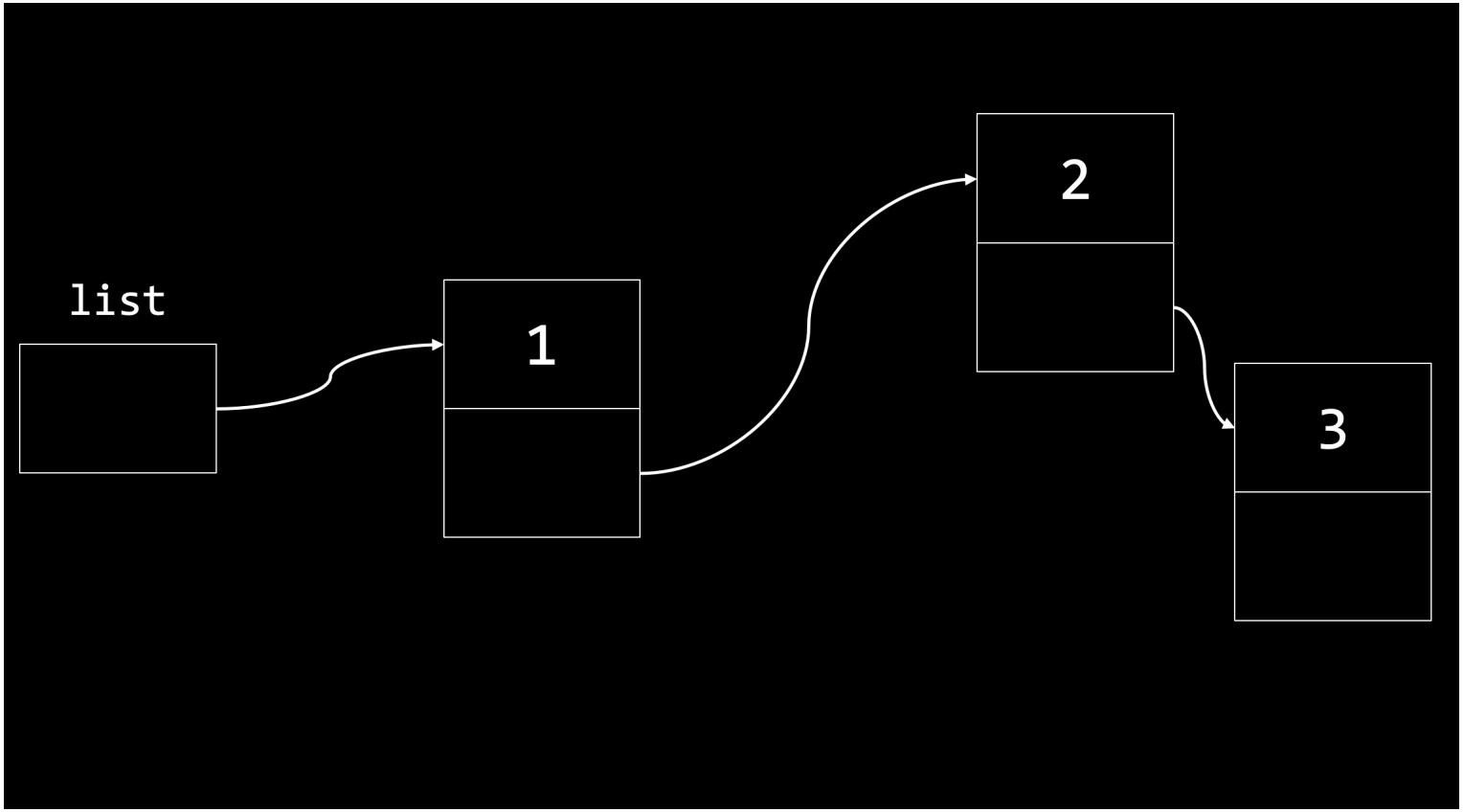
lista encadeada na memória

```
list->next->next = n;
```

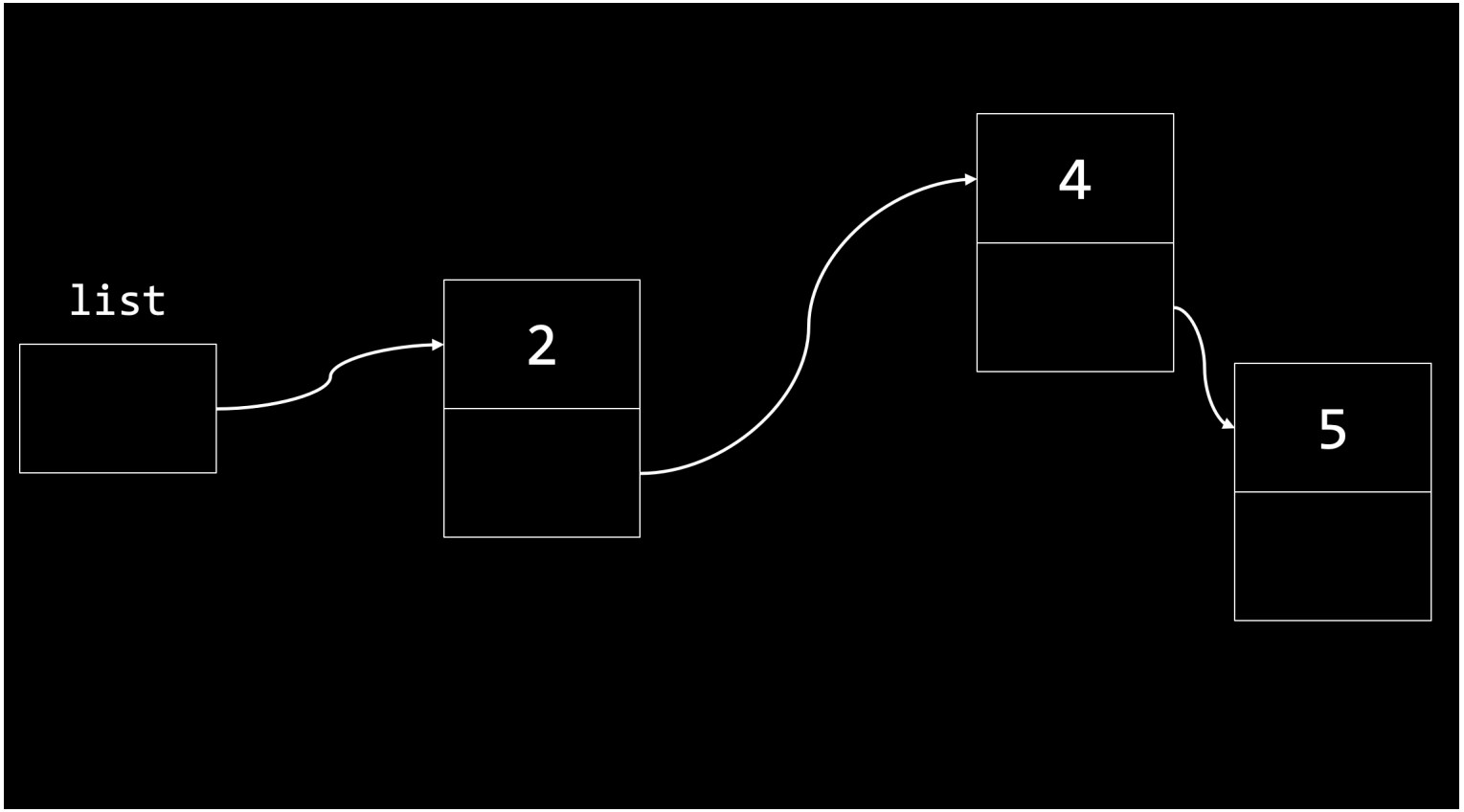
lista encadeada na memória



lista encadeada na memória



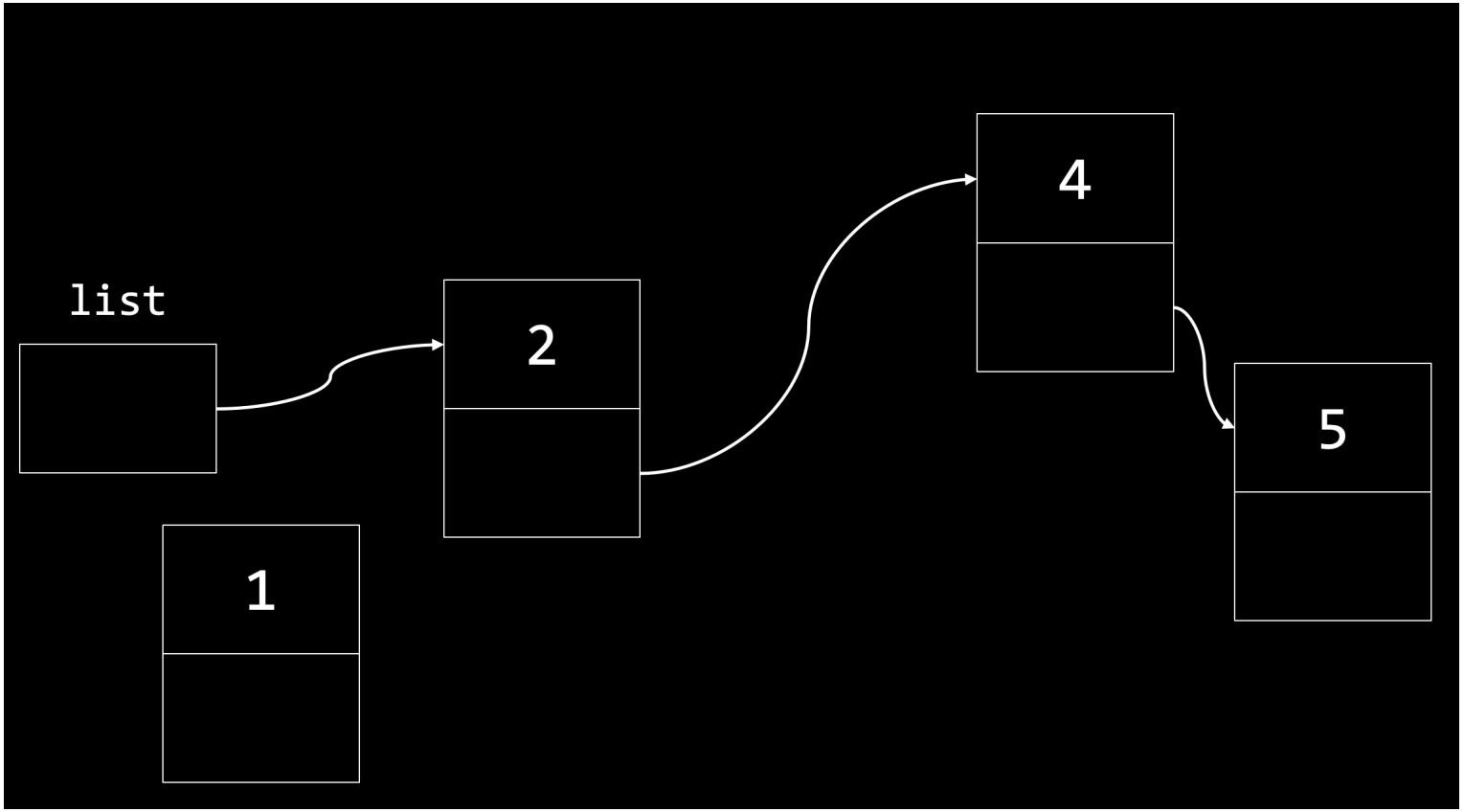
como **não** fazer



como **não** fazer

```
node *n = malloc(sizeof(node));
if (n != NULL)
{
    n->number = 1;
    n->next = NULL;
}
```

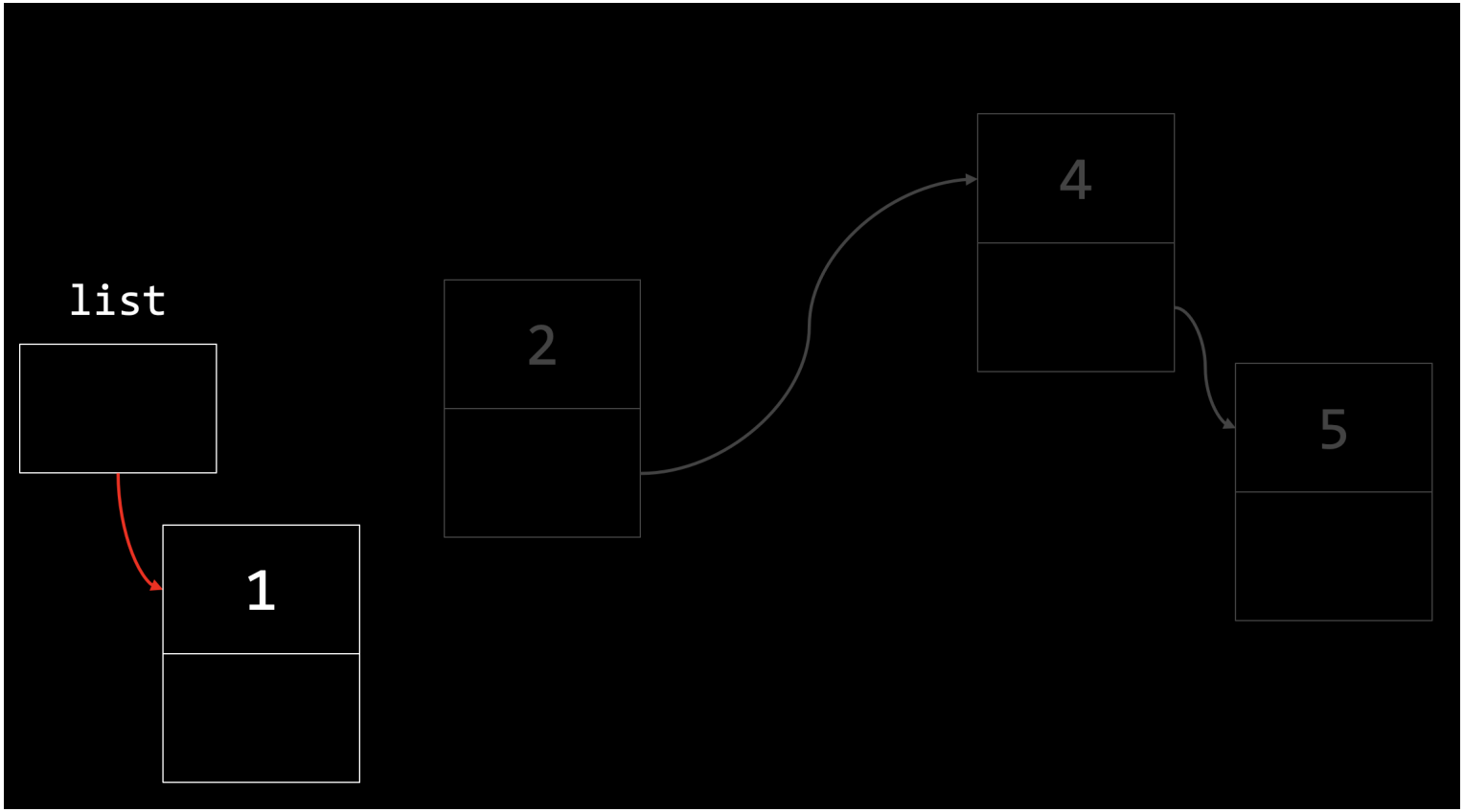
como **não** fazer



como **não** fazer

```
list = n;
```

como **não** fazer

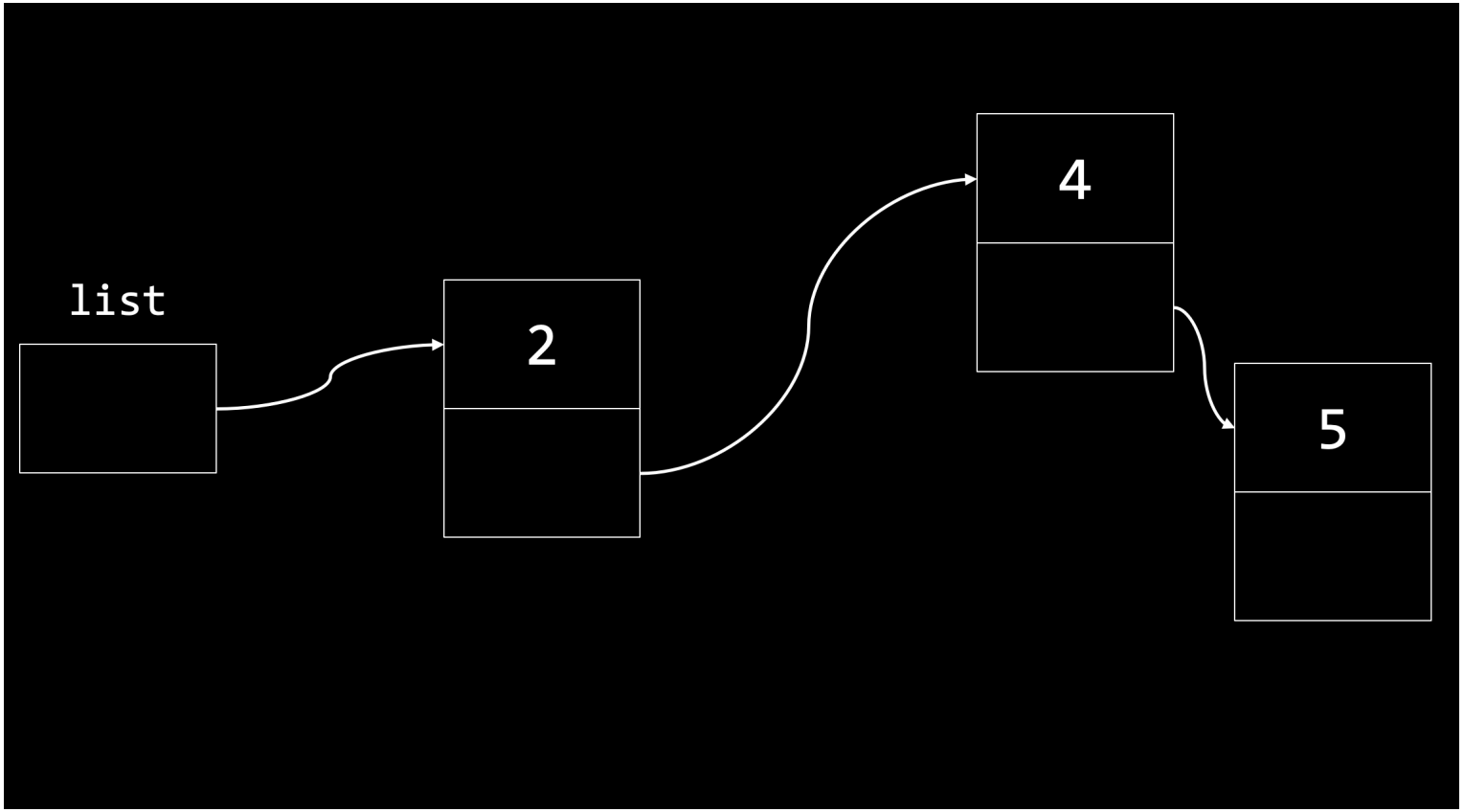


como **não** fazer

Não podemos atualizar o ponteiro `list` antes de armazenar seu valor em `n->next`.

adicionando elementos no início

adicionar um elemento no início tem custo $O(1)$



adicionando elementos no início

adicionar um elemento no início tem custo $O(1)$

```
node *n = malloc(sizeof(node));
if (n != NULL)
{
    n->number = 1;
    n->next = NULL;
}
```

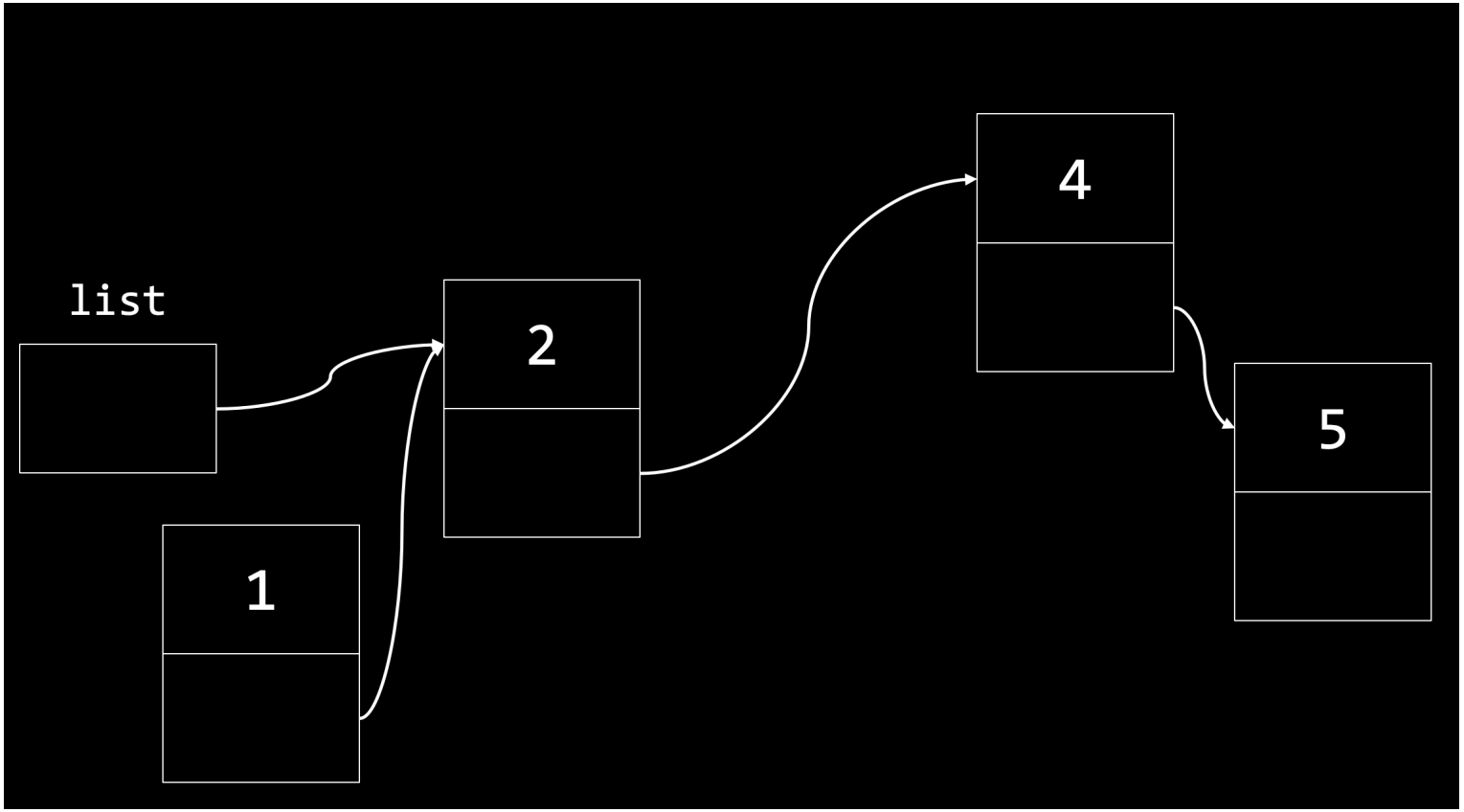
adicionando elementos no início

adicionar um elemento no início tem custo $O(1)$

```
n->next = list;
```

adicionando elementos no início

adicionar um elemento no início tem custo $O(1)$



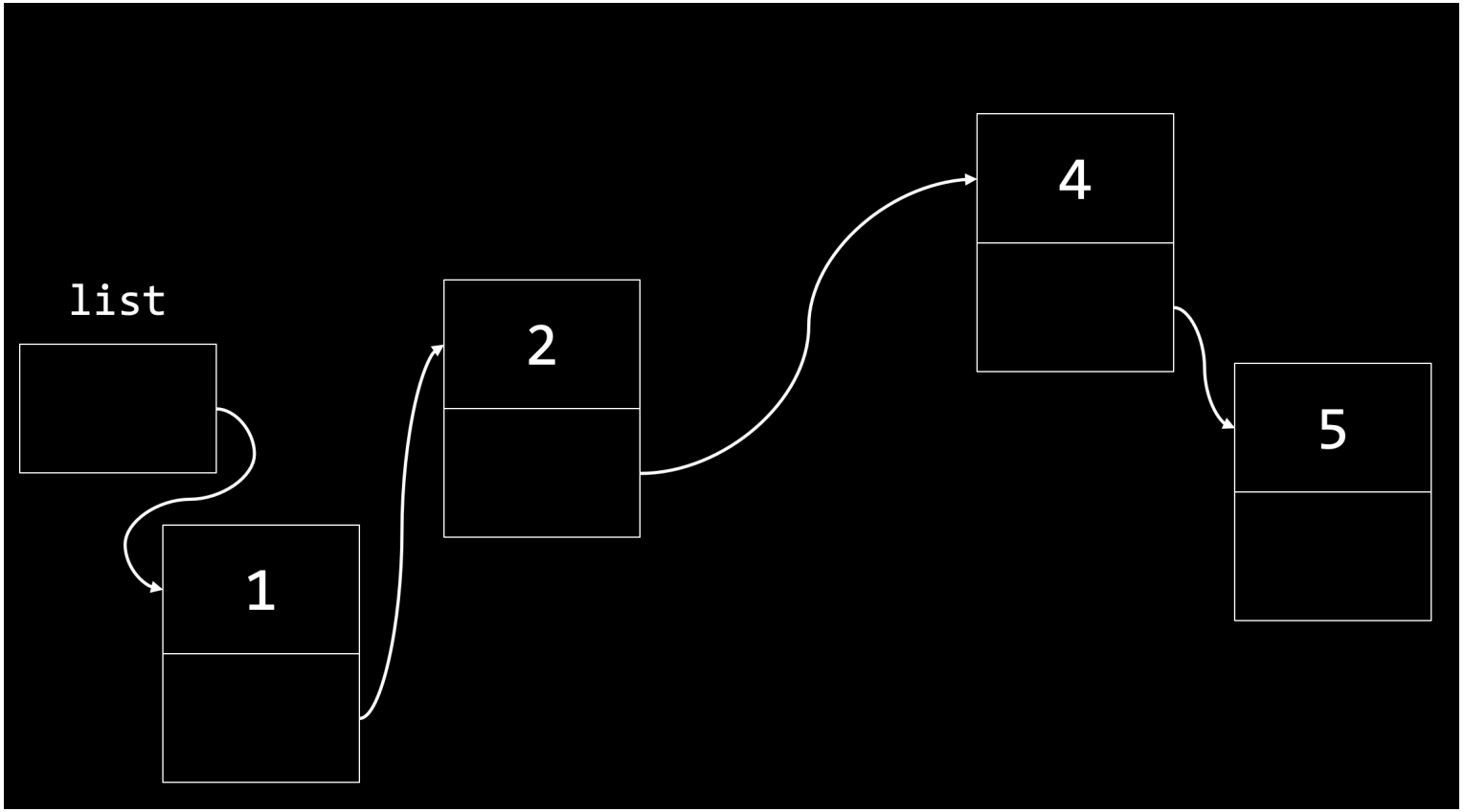
adicionando elementos no início

adicionar um elemento no início tem custo $O(1)$

```
list = n;
```

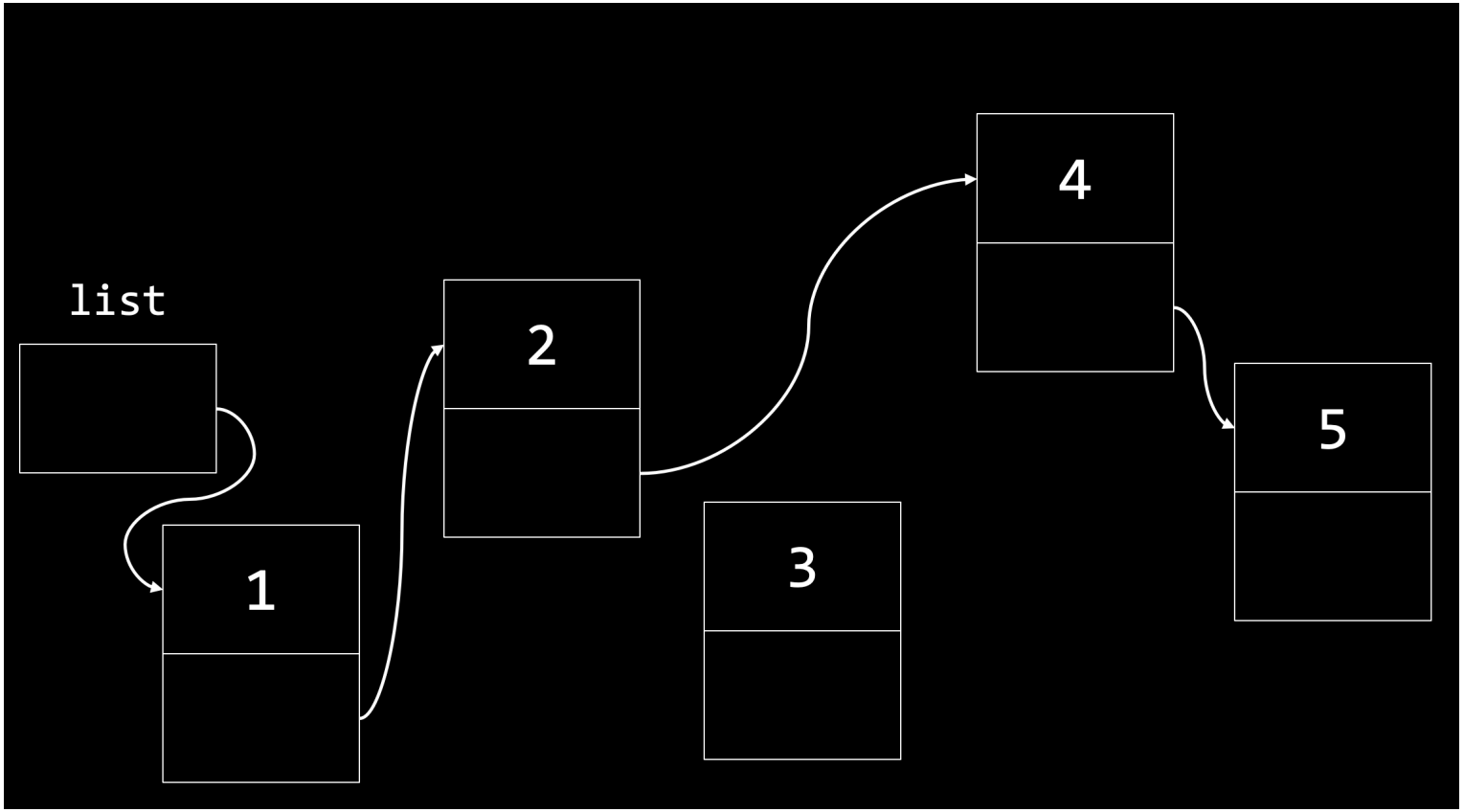
adicionando elementos no início

adicionar um elemento no início tem custo $O(1)$



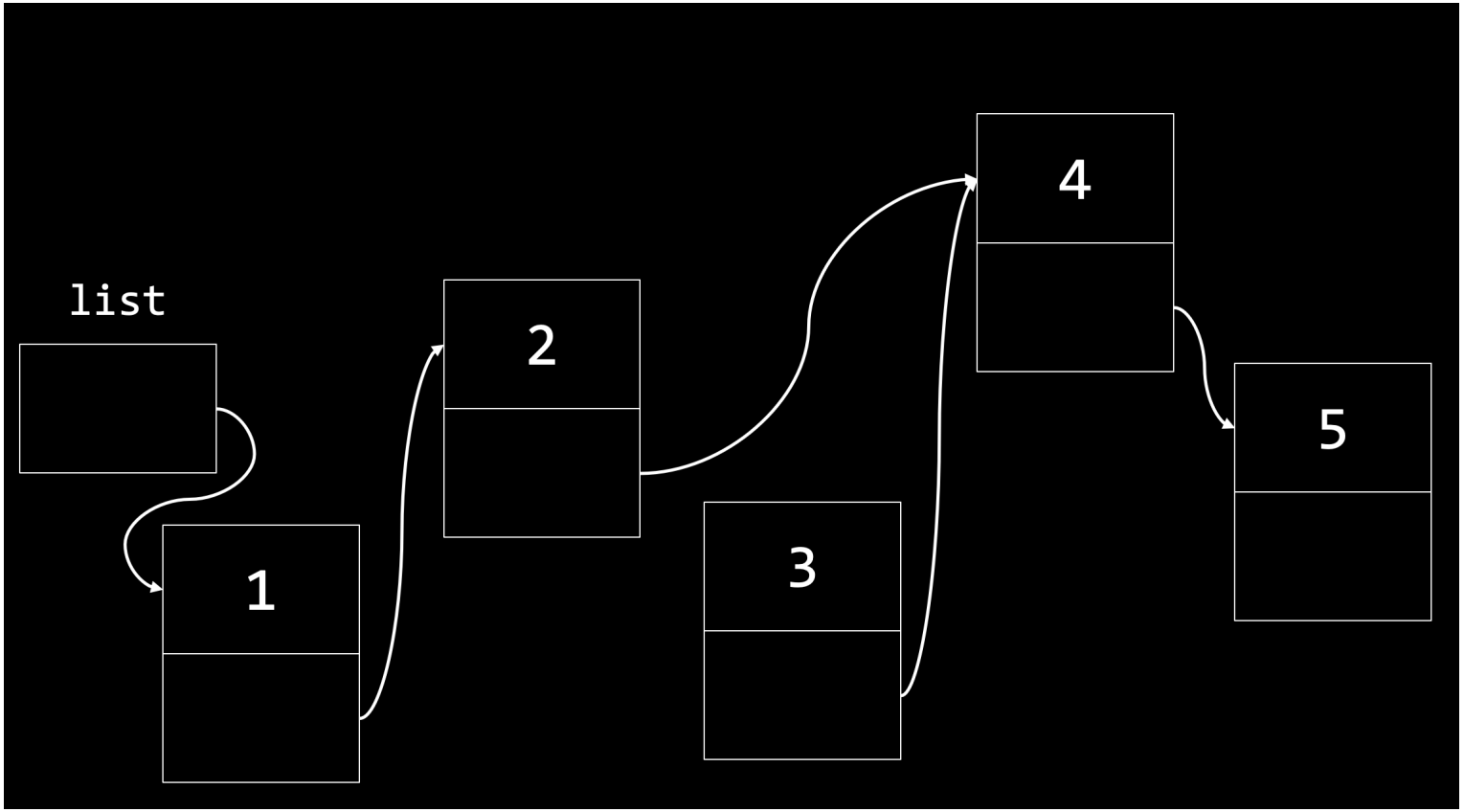
adicionando elementos em uma posição qualquer

após localizar o elemento, eventualmente $O(n)$, custo de $O(1)$



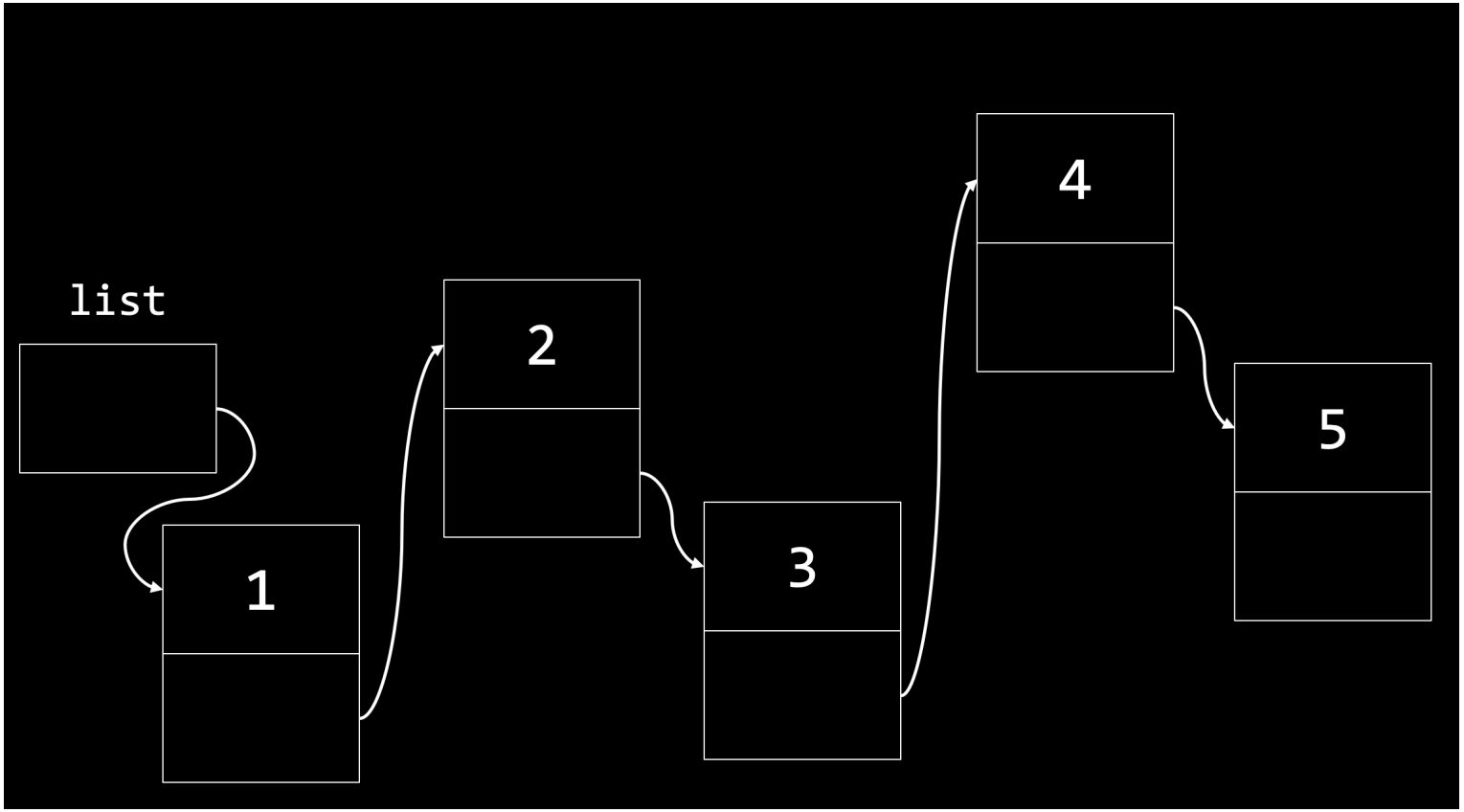
adicionando elementos em uma posição qualquer

após localizar o elemento, eventualmente $O(n)$, custo de $O(1)$

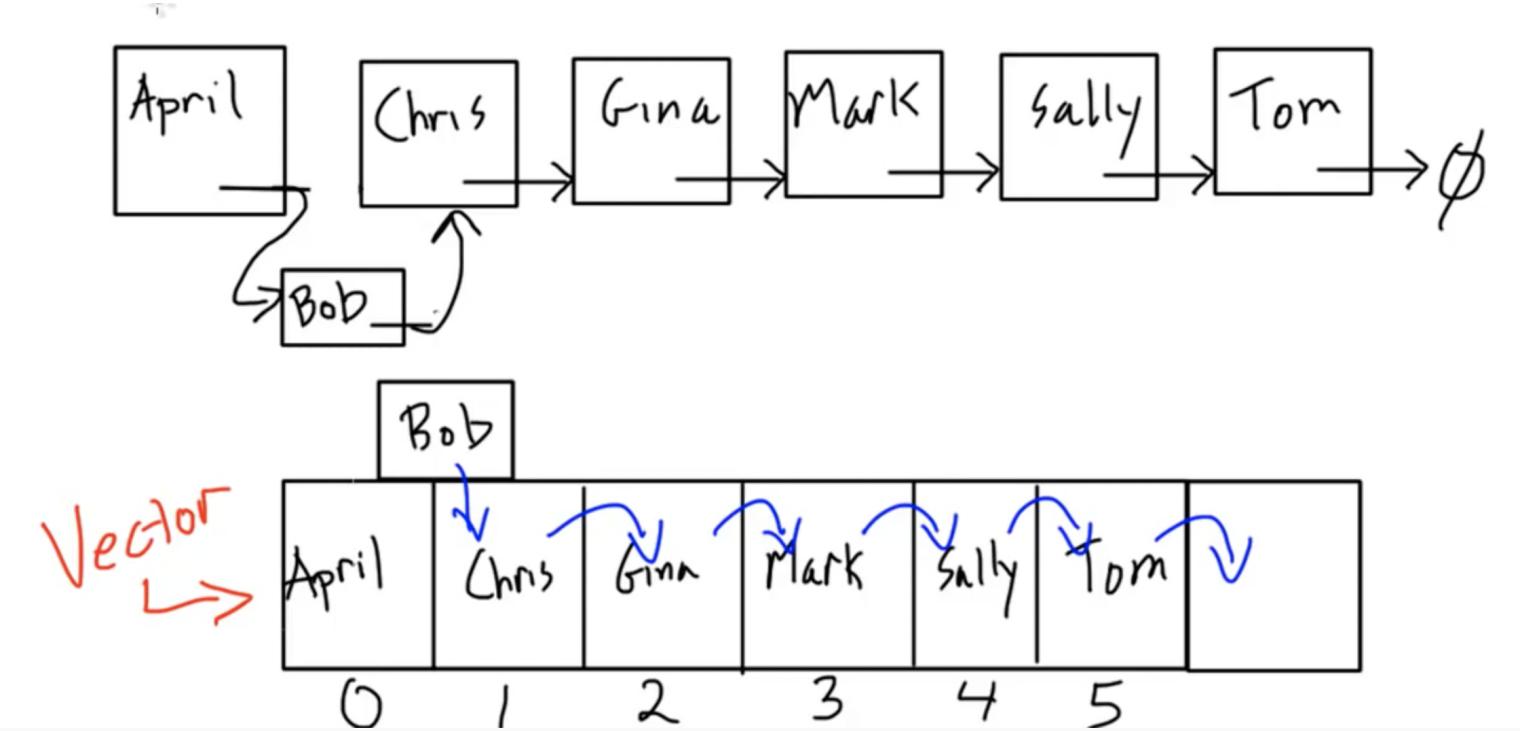


adicionando elementos em uma posição qualquer

após localizar o elemento, eventualmente $O(n)$, custo de $O(1)$



Sumário I



arrays também são chamados **vectors**.

Sumário II

complexidade deve ser avaliada para cada operação

- para recuperar o elemento da posição n?
  R: array $O(1)$, lista $O(n)$
- inserir elemento no início
  R: array $O(n)$, lista $O(1)$
- inserir elemento no final
  R: array $O(1)/O(n)$, lista $O(n)$
- inserir elemento no meio da estrutura?
- busca de um elemento na estrutura não ordenada?
- busca de um elemento na estrutura ordenada?